Lavaneko CPU 架构与指令集

16 位为默认版本，32 位为同构扩展版本。
围绕四个通用寄存器、主加减器、独立硬件乘除单元和 ROM 控制字构成的小型 CPU。

取指 PC / ROM / IR → 译码 控制字段 → 计算 寄存器 / ALU → 访存与写回 RAM / 选择器

项目	16 位版本	32 位版本
数据通路 / 寄存器宽度	16 位	32 位
通用寄存器	4 个：REG1 至 REG4	4 个：REG1 至 REG4
PC 与地址宽度	16 位，按字寻址	16 位，按字寻址
ROM 指令字宽度	32 位 = 16 位立即数 + 16 位控制	48 位 = 32 位立即数 + 16 位控制
程序 ROM 容量	65,536 个指令字	65,536 个指令字
数据 RAM 容量	65,536 × 16 位	65,536 × 32 位
状态标志	Z：零；C：进位 / 无借位	与 16 位版本一致

阅读路线：第 2 页看数据与时序，第 3 页看指令位图，第 4 页查指令语义，第 5 页手算编码，第 6 页看基础演示，第 7 至 8 页看乘除法扩展，第 9 页查文件位置，第 10 页看架构总览。

数据通路与执行时序



主图按流程从左到右排列。RAM 使用立即数低 16 位作为地址；ALU 结果可以直接进入写回选择器，不必先经过 RAM。

模块	主要职责
ProgramCounter	正常执行后 PC + 1；跳转时装载 ADDRESS；停机或复位时禁止普通更新。
RegisterBank	SELECT_A、SELECT_B 分别选择两个源寄存器；DEST 单独选择写入目标。读写不按寄存器编号自动轮换。
AddSub	A 选择立即数或读口 A；B 选择读口 B 或零。SUB=0 时 A+B，SUB=1 时 A-B。
DataRAM / 写回	STORE 写入读口 B 的值；LOAD 选择 RAM 读数写回。普通运算写回 ALU 结果。
Sequencer	组合提交与跳转条件，保存 Z/C；乘除期间保持 IR/PC，完成后统一写回。

一个完整时钟周期

时刻	动作
上升沿：取指	CPU 非忙碌时 IR 锁存 ROM[PC]；乘除等待期间保持 IR。随后译码与数据组合传播。
下降沿：提交	普通指令更新目标寄存器、RAM、标志与 PC；乘除先启动和迭代，再提交。JZ 使用已保存的 Z。
HALT / RESET	HALT 指令被取入后抑制后续更新，PC 保持在 HALT 地址；RESET 清零 PC、IR、寄存器、标志、乘除状态和仿真 RAM。

指令字：每一位控制什么

令数据宽度 W 为 16 或 32。每条指令由高 W 位立即数 I 和低 16 位控制字段组成。普通指令采用直接控制位；当位 7、6 同为 1，低 8 位改作扩展 opcode，见第 7 页。

高 W 位：I
16 位机为 [31:16]；32 位机为 [47:16]

→

低 16 位：CONTROL
位 [15:0]，包含读写选择与操作控制

位号	字段	置 1 或字段取值的含义
0	SUB	0：A+B；1：A-B
1	REG_WRITE	允许写入 DEST 指定的寄存器
2	LOAD	写回数据选择 RAM；0 时选择 ALU
3	STORE	允许把读口 B 的值写入 RAM[ADDRESS]
4	JUMP	无条件跳转至 ADDRESS
5	B_ZERO	ALU 的 B 输入为零；0 时为读口 B
6	HALT / EXT	位 7=0 时表示停机；位 7=1 时进入扩展类
7	A_REG / EXT	普通指令选择寄存器 A；与位 6 同为 1 时扩展
9:8	SELECT_B	读口 B 编号，00/01/10/11 对应 REG1/2/3/4
11:10	DEST	目标寄存器编号，映射同上
13:12	SELECT_A	读口 A 编号，映射同上
14	FLAGS_WRITE	将本次 ALU 的 Z、C 保存至标志寄存器
15	JZ	保存的 Z=1 时跳转至 ADDRESS

ADDRESS = I 的低 16 位。PC 每加 1 前进一整条指令，不是一个字节。ROM 和 RAM 是分离的地址空间。未使用的寄存器字段建议写 00。

编码公式：指令字 = (I << 16) | BASE | (B << 8) | (D << 10) | (A << 12)。I 保留低 W 位；A、B、D 为 0 至 3 的编号。

指令集速查

下表使用易读的汇编式记法。Rd 为目标，Ra/Rb/Rs 为源，#I 为立即数，[addr] 为 RAM 地址。

指令记法	BASE	执行效果	Z / C
NOP	0000	只前进 PC	保持
LDI Rd, #I	0022	Rd = I；选择 B=0	保持
ADDI Rd, Rs, #I	4002	Rd = I + Rs；Rs 编入 B 字段	更新
ADD Rd, Ra, Rb	4082	Rd = Ra + Rb	更新
SUB Rd, Ra, Rb	4083	Rd = Ra - Rb	更新
CMP Ra, Rb	4081	计算 Ra - Rb，只写标志	更新
LOAD Rd, [addr]	0006	Rd = RAM[addr]	保持
STORE Rs, [addr]	0008	RAM[addr] = Rs；Rs 编入 B 字段	保持
JMP addr	0010	PC = addr；编码 API 名为 jump	保持
JZ addr	8000	Z=1 时 PC=addr，否则 PC+1	保持
HALT	0040	停机，PC 保持在当前指令地址	保持

BASE 不是整条机器码

BASE 是尚未加入寄存器编号和立即数的低位控制掩码。例如 ADD REG3, REG1, REG2 的低 16 位是 0x4982，而不是 0x4082。

Z=1 表示截断为 W 位后的运算结果为零。加法 C=1 表示超出 W 位；减法与 CMP 的 C=1 表示没有借位（无符号 A ⩾ B）。没有有符号溢出 V 或负数 N 标志。

运算按模 2^W 截断。负数可用 W 位补码表示，但不能把 C 当作有符号大小比较结果。当前条件分支只有 JZ，没有 JC、JN 或 JNZ。

手写机器码：从字段到 0 和 1

例 1：把十进制 10 装入 REG1

LDI REG1, #10：I=0x000A，D=00；置 REG_WRITE 和 B_ZERO。
16 位版本机器码：0x000A0022
二进制：`0000 0000 0000 1010 | 0000 0000 0010 0010`

例 2：REG3 = REG1 + REG2

字段	数值	贡献
BASE	ADD	0x4082
SELECT_A	REG1 = 00	0x0000
SELECT_B	REG2 = 01	0x0100
DEST	REG3 = 10	0x0800
I	0；本条使用寄存器 A	0x00000000

合并得到 0x00004982。低 16 位二进制：
`0100 1001 1000 0010`
下降沿只改写 REG3，同时更新 Z、C；REG1、REG2、REG4 保持原值。

例 3：同一控制格式扩展到 32 位

LDI REG4, #0x12345678：D=11，使控制字段为 0x0C22。
48 位机器码：0x123456780C22。
32 位版本仍使用 16 位 PC 与 RAM 地址，LOAD/STORE/JMP/JZ 只读取立即数低 16 位。

减法复用加法器
A-B = A + 按位取反(B) + 1。SUB 同时选择取反后的 B，并将最低位进位输入置 1。16 位版本级联四个 4 位加法器，32 位版本级联八个；最高位进位作为“无借位”标志。

基础 ROM 示例与预期结果

以下基础示例保留在 programs/legacy16.hex。地址与机器码用十六进制，普通数字用十进制。默认 ROM 还包含乘除演示，见第 8 页。

地址	机器码	汇编式说明
0000	000A0022	LDI REG1, #10
0001	00030422	LDI REG2, #3
0002	00004982	ADD REG3, REG1, REG2
0003	00006D83	SUB REG4, REG3, REG2
0004	00200208	STORE REG3, [0x20]
0005	00200406	LOAD REG2, [0x20]
0006	00005281	CMP REG2, REG3
0007	000A8000	JZ 10
0008	DEAD0022	LDI REG1, #0xDEAD（被跳过）
0009	00000040	HALT（被跳过）
000A	00050022	LDI REG1, #5
000B	00000422	LDI REG2, #0
000C	00010C22	LDI REG4, #1
000D	00005482	ADD REG2, REG2, REG1
000E	00004383	SUB REG1, REG1, REG4
000F	00128000	JZ 18
0010	000D0010	JMP 13
0011	00000040	HALT（正常路径不执行）
0012	00210108	STORE REG2, [0x21]
0013	00210806	LOAD REG3, [0x21]
0014	00000040	HALT

流程：10+3 → 减法校验 → RAM 往返 → 相等跳转 → 循环求 5+4+3+2+1 → 保存结果并停机。

最终可见状态

PC=0x0014；REG1=0；REG2=15；REG3=15；REG4=1；Z=1；C=1；halt=1。RAM[0x20]=13，RAM[0x21]=15。

乘除法扩展指令

控制位 7、6 同为 1 时，低 8 位改为扩展操作码。A/B/D 寄存器字段位置保持不变；位 15:14 与高 W 位立即数忽略，编码时写零。扩展类不会触发普通 HALT、RAM、跳转或标志写入。

指令记法	低 8 位	结果
MULU Rd, Ra, Rb	C0	无符号乘法；HI:LO 保存完整乘积，Rd=LO
MUL Rd, Ra, Rb	C1	有符号乘法；HI:LO 保存补码乘积，Rd=LO
DIVU Rd, Ra, Rb	C2	无符号除法；LO 为商，HI 为余数，Rd=LO
DIV Rd, Ra, Rb	C3	有符号除法；LO 为商，HI 为余数，Rd=LO
MFLO Rd	C4	Rd=LO
MFHI Rd	C5	Rd=HI
MDSTAT Rd	C6	错误码零扩展后写入 Rd

这七条指令均保持 Z、C。HI 与 LO 各宽 W 位，在乘除完成时一起更新，并保持到下一次乘除完成或复位。乘法保留 2W 位结果，因此不报告乘法溢出；Rd 只接收低 W 位。

MD_ERROR	含义	提交行为
0	正常完成	写 Rd、HI、LO；PC 前进
1	除数为零	Rd 保持；HI=LO=0；PC 前进
2	有符号 MIN / -1	Rd 保持；HI=LO=0；PC 前进

MIN 为最小有符号数：16 位为 -32768，32 位为 -2147483648。下一次正常乘除会清除错误码；MFLO、MFHI、MDSTAT 只读取，不清除状态。其他扩展 opcode 保留，当前作为 NOP 执行。

编码例：MUL REG3, REG1, REG2
0xC1 | (0b01 << 8) | (0b10 << 10) | (0b00 << 12) = 0x09C1。
16 位版本完整指令为 0x000009C1；低字节二进制为 1100 0001。
若 REG1=-7、REG2=3，结果为 HI=FFFF、LO=FFEB，REG3=FFEB。

独立乘除硬件与执行过程

乘法器和除法器拥有各自的加减电路、寄存器与计数器，不占用主 ALU。内部由可展开的加法器、选择器、寄存器和接线构成。数据运算的加减电路使用 4 位加法器级联。

乘法：每轮决定是否累加，然后右移

先取两个操作数的幅值：M=|A|，Q=|B|，高位累加器 H=0。每轮检查 Q 的最低位：为 1 则 H 加 M，否则加零；将进位:H:Q 整体右移一位。重复 W 轮后，H:Q 得到完整乘积。有符号模式按两输入符号异或，对整个 2W 位乘积条件取负。

除法：左移、试减、选择余数

设 Q=|被除数|，D=|除数|，W+1 位余数 R=0。每轮把 R:Q 左移一位，再计算 R-D。没有借位时保存差值并把新商位设为 1；有借位时选择试减前的 R，新商位为 0。重复 W 轮即可得到商与余数；恢复通过选择器完成，不另占一个加回周期。

有符号除法约定

商向零截断，余数符号随被除数。例如 -7÷3 得商 -2、余数 -1；7÷(-3) 得商 -2、余数 1。正常结果始终满足：被除数 = 除数×商 + 余数。

阶段	下降沿次数	CPU 行为
启动	1	锁存操作数与符号；BUSY=1；保持 PC/IR
迭代	W	每轮处理一位；通用寄存器和标志保持
完成提交	1	写回结果与状态、PC+1、清 BUSY
除零 / 除法溢出	共 2	启动后直接报告错误，下一个下降沿提交

正常乘除：16 位为 18 个执行周期，32 位为 34 个。完成前不会取入下一条指令；支持紧接着执行下一次乘除。RESET 可中止当前运算并清零全部状态。

默认演示 ROM 的最终状态

程序先运行基础加减、RAM 和循环，再执行乘除、高低结果读取及两类错误处理，最后计算 7÷(-3)。PC=0x002F；REG1=7，REG2=-3，REG3=-2，REG4=1；LO=-2，HI=1，MD_ERROR=0，BUSY=0，Z=C=1，halt=1。

使用方法与文件位置

打开与运行

在项目目录执行 `npm run open` 打开 16 位版本，或用 `npm run open:32` 打开 32 位版本。选中 Lavaneko 主电路，点击 RESET，使用 Simulate 菜单的 Manual Tick Full Cycle 推进一个完整周期；乘除需要多个周期；Auto-Tick Enabled 可开启自动时钟。

位置（相对于项目根目录）	用途
apps/logisim/Lavaneko16.circ	默认 16 位仿真电路
apps/logisim/Lavaneko32.circ	同构的 32 位仿真电路
packages/isa/lib/architecture.js	指令掩码、编码函数和参考执行模型
packages/circuits/lib/	实际生成各模块与顶层接线的源文件
programs/demo16.hex / demo32.hex	包含乘除指令的默认 ROM 镜像
programs/legacy16.hex / legacy32.hex	第 6 页的基础 ROM 示例
docs/Lavaneko-架构总览.svg / .png	可编辑矢量图与高清架构图
docs/architecture.md	架构与模块边界说明
docs/reference/legacy-cpu.jpg	用户提供的旧架构截图

修改 packages/circuits 中的电路定义或 packages/isa 中的指令定义后，执行 npm run build 重新生成电路与 ROM 镜像。

LAVANEKO

CPU 架构总览

